A Second Course in Algorithms

Mihir Rao

Contents

1 Course Goals and Introduction to Maximum Flow **2**
1.1 Maximum flow 2
1.2 Why greedy paths can fail 2
1.3 Ford–Fulkerson and optimality 3

Lecture 1: Course Goals and Introduction to Maximum Flow

1.1 Maximum flow

A flow network is a directed graph $G = (V, E)$ with source s, sink t, and nonnegative integer capacities u_e on its edges. A FLOW assigns f_e to each edge, subject to $0 \le f_e \le u_e$ and conservation at every vertex other than s and t: incoming flow equals outgoing flow. Assume no edges enter s or leave t. The VALUE $|f|$ is the flow leaving s; conservation makes it equal to the flow entering t. The goal is a feasible flow of maximum value. This models a steady rate through a network, rather than the arrival time of individual units.

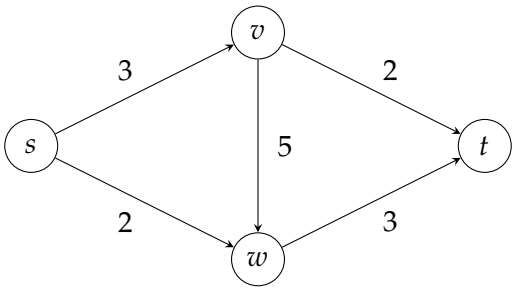


Figure 1: Edge labels are capacities. The maximum flow has value 5.

In the network above, send two units along $s \to v \to t$, two along $s \to w \to t$, and one along $s \to v \to w \to t$. This is optimal: only five units can leave s. Flow also models less literal problems such as matching, where a reduction translates feasible solutions and objectives into a network.

1.2 Why greedy paths can fail

A natural algorithm repeatedly sends as much additional flow as possible along an unsaturated s–t path. In the example, choosing $s \to v \to w \to t$ first sends three units and saturates both $s \to v$ and $w \to t$. No entirely unsaturated forward path remains, yet the flow has value only three. The problem is that an early path choice may need to be partially undone.

For a current flow f, the RESIDUAL NETWORK has, for each original edge $e = (v, w)$, a forward residual edge of capacity $u_e - f_e$ and a backward residual edge (w, v) of capacity f_e. Zero-capacity edges are omitted. Backward capacity represents the amount of existing flow that may be canceled; oppositely directed original edges are treated as distinct residual edges. In the example, after the bad greedy choice, the residual path $s \to w \to v \to t$ sends two units forward on its outer edges and cancels two on $v \to w$, reaching value five.

1.3 Ford–Fulkerson and optimality

The Ford–Fulkerson algorithm starts with zero flow and repeatedly finds an s–t path of positive residual capacity. It augments by the minimum residual capacity on that path, increasing original flow on forward edges and reducing it on backward edges. It stops when no such path remains.

Each augmentation preserves capacities by the bottleneck choice and preserves conservation at every intermediate vertex. With integer capacities, the flow and all residual capacities remain integral, so each augmentation increases the value by at least one. The source capacity bounds the value, hence the algorithm terminates. A graph search takes $O(|E|)$ per augmentation; this gives a bound proportional to the maximum flow value, which need not be polynomial in the bit length of the capacities.

The structural optimality condition is: *if the residual network contains no s–t path, the current flow is maximum*. Ford–Fulkerson terminates only when this condition holds. This separates proving that a condition certifies optimality from designing an algorithm that reaches it; the proof of the condition and the max-flow/min-cut theorem follow in the next lecture.

Source. Tim Roughgarden, A Second Course in Algorithms, Winter 2016, Lecture 1.